

Identify and debug

1. Course Content

Note: This program runs on a sandbox map. Personal maps require manual program adjustments. This course focuses on debugging the confidence level of the car in recognizing each road sign.

Basic: Start the autonomous driving program and observe the road sign recognition results.

2. Pre-Test Preparation

- Adjust model parameters

Parameter path: `/home/jetson/yahboomcar_auto/src/auto_driver/config/yolo_param.yaml`

```
#Road sign detection inference parameters
sign_detection:
  model_path: '/home/jetson/yahboomcar_auto/src/sign_model.engine'
  conf: 0.92    #Set the minimum confidence threshold for detection. Detected
               #objects with confidence below this threshold will be ignored. Adjusting this
               #value helps reduce false positives.
  iou: 0.7     #Overlap threshold for Non-Maximum Suppression (NMS). Lower
               #values reduce detection results by eliminating overlapping boxes
  rect: True   #If enabled, applies minimal padding to the shorter side of the
               #image until it's divisible by stride, improving inference speed.
  half: True   #If enabled, uses half-precision inference, which can
               #accelerate model inference speed on supported GPUS with minimal impact on
               #accuracy.
  device: "cuda:0"    #Inference device
  batch : 50          #Batch size, larger batch sizes can provide higher
               #throughput, thus reducing total inference time.
  max_det : 2         #Maximum number of objects to detect
  augment : False     #Enable Test Time Augmentation (TTA) for prediction, which
               #may improve detection robustness but reduces inference speed.
  verbose : False     #Whether to output detailed inference information in
               #terminal.
  classes : [0, 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 12, 13]

line_detection:
#Lane detection inference parameters
  model_path: '/home/jetson/yahboomcar_auto/src/lane_v6.engine'
  conf: 0.5    #Set the minimum confidence threshold for detection. Detected
               #objects with confidence below this threshold will be ignored. Adjusting this
               #value helps reduce false positives.
  iou: 0.6     #Overlap threshold for Non-Maximum Suppression (NMS). Lower
               #values reduce detection results by eliminating overlapping boxes
  rect: True   #If enabled, applies minimal padding to the shorter side of the
               #image until it's divisible by stride, improving inference speed.
  half: True   #If enabled, uses half-precision inference, which can
               #accelerate model inference speed on supported GPUS with minimal impact on
               #accuracy.
  device: "cuda:0"    #Inference device
```

```
batch : 50          #Batch size, larger batch sizes can provide higher
throughput, thus reducing total inference time.
max_det : 4         #Maximum number of objects to detect
augment : False     #Enable Test Time Augmentation (TTA) for prediction, which
may improve detection robustness but reduces inference speed.
verbose : False     #Whether to output detailed inference information in
terminal.
```

sign_model.engine: Road sign recognition model

lane_V6.engine: Lane detection recognition model

For normal road sign detection debugging, you only need to change two parameters: conf: 0.92 minimum confidence threshold, which means objects with similarity higher than 0.92 will be boxed. max_det: 2 maximum number of objects to detect, how many objects can be detected simultaneously at the same time. For detailed understanding of other parameters, you can refer to the comments.

- Adjust test parameters

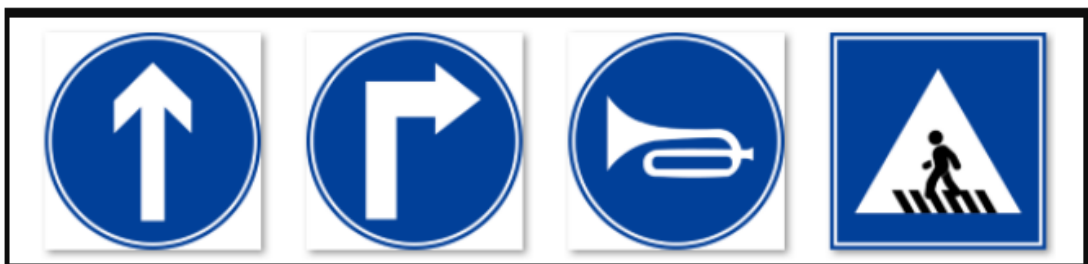
Parameter path:

```
/home/jetson/yahboomcar_auto/src/auto_driver/config/auto_drive_node.yaml
```

```
turn_radius: 0.4
linear_speed: 0.13
angular_speed: -0.55
duration: 2.0
response_distance_1: 2900
response_distance_2: 132
cooldown_time: 0.2
DEBUG_MODE: True
DEBUG_MODE_2: False
START_AutoDrive: False
Kp: -0.62
Ki: 0.0
Kd: -0.1
max_integral: 1000.0
image_size: [640, 480]
```

- Prepare road signs

Straight, right turn, horn, sidewalk



Speed limit, stop, school, parking lot B



Traffic lights, parking lot A



3. Run Example Program

Autonomous driving source code path:

```
/home/jetson/yahboomcar_auto/src/auto_driver/scripts/auto_drive.py
```

- Start camera + chassis + autonomous driving node (need to run in virtual environment terminal)

```
roslaunch auto_driver auto_drive.launch
```

```

(yolov11) jetson@yahboom:~$ roslaunch auto_driver auto_driver.launch
... logging to /home/jetson/.ros/log/2add7f36-d0c0-11f0-9d82-48b02d5ab21e/roslaunch-yahboom-15809.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

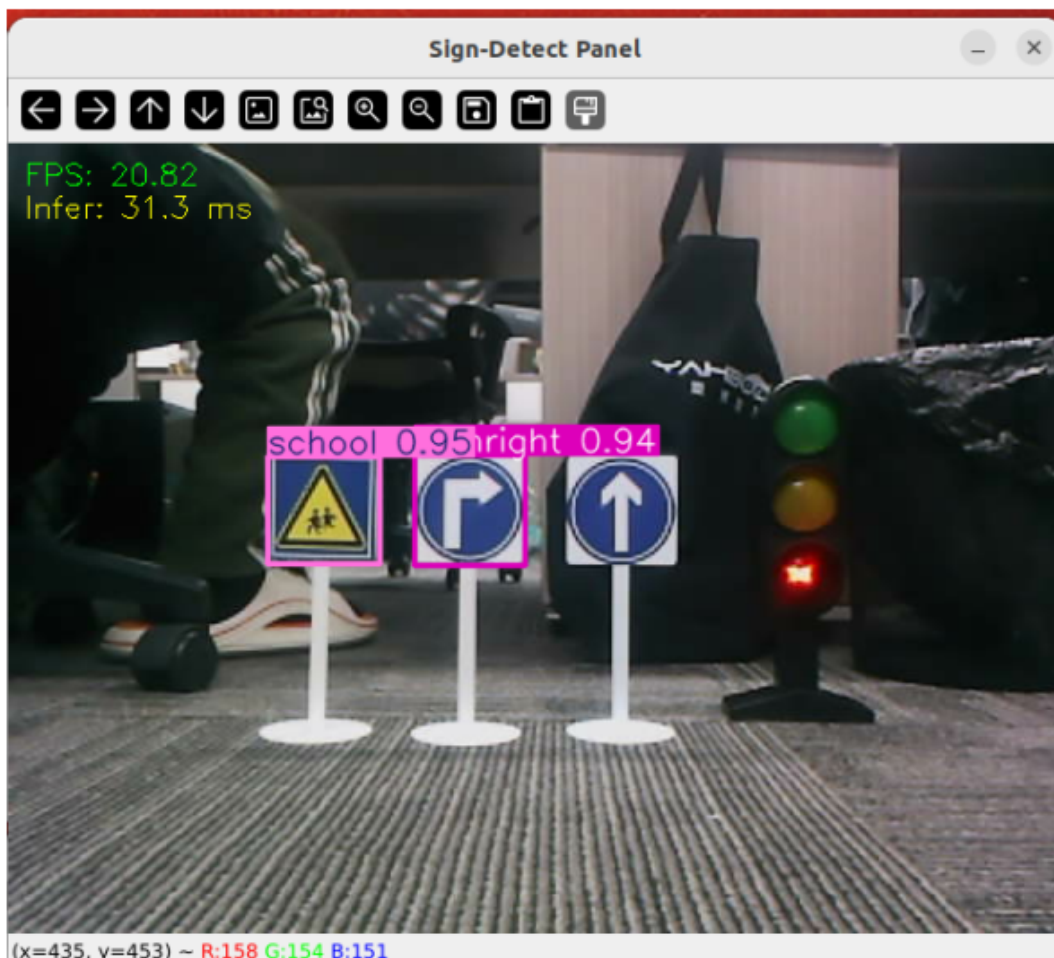
started roslaunch server http://yahboom:44059/

SUMMARY
=====
PARAMETERS
* /ascamera_hp60c/color_pcl: False
* /ascamera_hp60c/confiPath: /home/jetson/yahb...
* /ascamera_hp60c/depth_height: 480
* /ascamera_hp60c/depth_width: 640
* /ascamera_hp60c/fps: 30
* /ascamera_hp60c/pub_mono8: False
* /ascamera_hp60c/pub_tfTree: True
* /ascamera_hp60c/rgb_height: 480
* /ascamera_hp60c/rgb_width: 640
* /ascamera_hp60c/usb_bus_no: -1
* /ascamera_hp60c/usb_path: null
* /auto_drive/DEBUG_MODE: False
* /auto_drive/DEBUG_MODE_2: True
* /auto_drive/Kd: -0.1
* /auto_drive/Ki: 0.0
* /auto_drive/Kp: -0.62
* /auto_drive/START_AutoDrive: False
* /auto_drive/angular_speed: -0.55
* /auto_drive/cooldown_time: 0.2
* /auto_drive/duration: 2.0
* /auto_drive/image_size: [640, 480]
* /auto_drive/linear_speed: 0.13
* /auto_drive/max_integral: 1000.0
* /auto_drive/response_distance_1: 2900
* /auto_drive/response_distance_2: 132
* /auto_drive/turn_radius: 0.4
* /auto_drive/turn_speed: 0.4

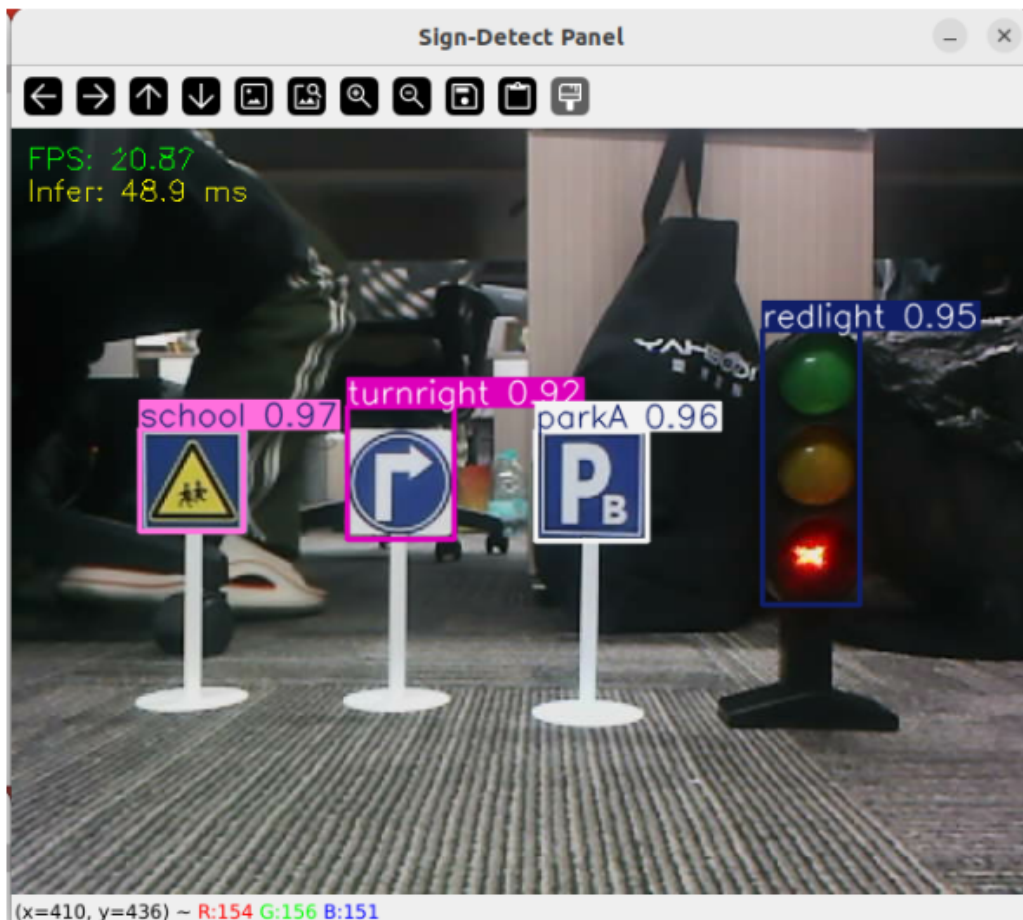
```

After successful startup, the display will not move to test road sign recognition. (If you want it to move, change the parameter START_AutoDrive in auto_drive_node.yaml to True)

Place road signs in the center of the screen to recognize the correct sign names. max_det: 2 recognizes only two road signs at the same time. False recognition may occur during autonomous driving, it is recommended to change to max_det: 2.



max_det: 4 recognizes four road signs at the same time



Ensure all road signs on the road can be recognized, then proceed to the next debugging step.

4. Source Code Analysis

4.1, auto_drive.py

Program source code path:

/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/auto_drive.py

YOLO processing flow (handle_sign):

- Get image frame
- Use YOLO model for object detection
- Process detection results:
 - Filter ignored signs (such as "turnright_ground", "sidewalk_ground")
 - Publish detected sign information
 - Display FPS and inference time in debug mode
 - Real-time display of detection results

The main thread is responsible for receiving images, while the worker thread handles image processing and detection tasks, communicating between threads through queues.

```
def _sign_detection_worker(self):
    '''Sign detection worker thread'''
    sign_pub_counter = 0

    while not rospy.is_shutdown() and not self._shutdown_flag:
        if self.cv_image is None:
```

```

        time.sleep(0.01)
        continue

    # Check cooldown status
    current_time = time.time()
    if self.sign_cooldown:
        if current_time - self.last_sign_time >= self.cooldown_duration:
            self.sign_cooldown = False
            rospy.loginfo("Sign detection cooldown ended")
        else:
            # During cooldown period, skip sign detection
            time.sleep(0.01)
            continue

    try:
        frame = self.cv_image.copy()
        infer_start = time.time()

        # Perform sign detection
        results = self.sign_detect.get_infer_result(frame, True)
        annotated_frame = frame.copy()

        # Record whether detected sign requires cooldown
        detected_sign_requires_cooldown = False

        for res in results:
            if res.bboxes is not None:
                annotated_frame = res.plot()
                bboxes = res.bboxes
                for i in range(len(bboxes)):
                    box_coords = bboxes.xyxy[i].tolist()
                    class_name = res.names[int(bboxes.cls[i].item())]
                    confidence = bboxes.conf[i].item()

                    # Check if it's an ignored sign
                    if class_name in self.ignored_signs:
                        # rospy.loginfo(f"Ignoring sign: {class_name}")
                        continue # Skip ignored signs

                    # Publish detection results
                    if sign_pub_counter % 5 == 0:
                        detection_msg = DetectionObject()
                        detection_msg.class_name = class_name
                        detection_msg.confidence = confidence
                        detection_msg.xmin = float(box_coords[0])
                        detection_msg.ymin = float(box_coords[1])
                        detection_msg.xmax = float(box_coords[2])
                        detection_msg.ymax = float(box_coords[3])
                        self.sign_publisher.publish(detection_msg)

                    # Add detection result to event queue (ignored
signs are not added)

                    if not self.action_running:
                        object_distance =
self._get_distance(float(box_coords[1]))
                        obj = Object(class_name, confidence,
object_distance)

                    try:

```

```

        self.event_queue.put_nowait(obj)
    except queue.Full:
        pass

    # Set cooldown flag (all signs except ignored
ones trigger cooldown)

    detected_sign_requires_cooldown = True

    sign_pub_counter += 1

    # If detected sign requires cooldown, start cooldown
    if detected_sign_requires_cooldown:
        self.sign_cooldown = True
        self.last_sign_time = current_time
        rospy.loginfo(f"Sign detected, starting
{self.cooldown_duration}s cooldown")

    # Calculate and display FPS
    if self.DEBUG_MODE_2:
        infer_end = time.time()
        self.frame_count_sign += 1
        curr_time = time.time()
        time_diff = curr_time - self.prev_time_sign
        if time_diff >= 1.0:
            self.fps_sign = self.frame_count_sign / time_diff
            self.frame_count_sign = 0
            self.prev_time_sign = curr_time

        cv2.putText(annotated_frame, f"Sign FPS:
{self.fps_sign:.2f}", (10, 65),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 1)
        infer_time = (infer_end - infer_start) * 1000
        cv2.putText(annotated_frame, f"Sign Infer: {infer_time:.1f}
ms", (10, 85),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 1)

    # Display cooldown status
    cooldown_status = "COOLDOWN" if self.sign_cooldown else
"READY"

    cooldown_color = (0, 0, 255) if self.sign_cooldown else (0,
255, 0)

    remaining_time = max(0, self.cooldown_duration -
(current_time - self.last_sign_time))
    cv2.putText(annotated_frame, f"Status: {cooldown_status}
({remaining_time:.1f}s)",
                (10, 105), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
cooldown_color, 1)

    # Display sign detection results
    if self.DEBUG_MODE_2:
        cv2.imshow("Sign-Detect Panel", annotated_frame)
        cv2.waitKey(1)

except Exception as e:
    rospy.logerr("Error in sign detection: %s", str(e))
    time.sleep(0.01)

```

